Executable proof of concept: BPMN calling a REST API

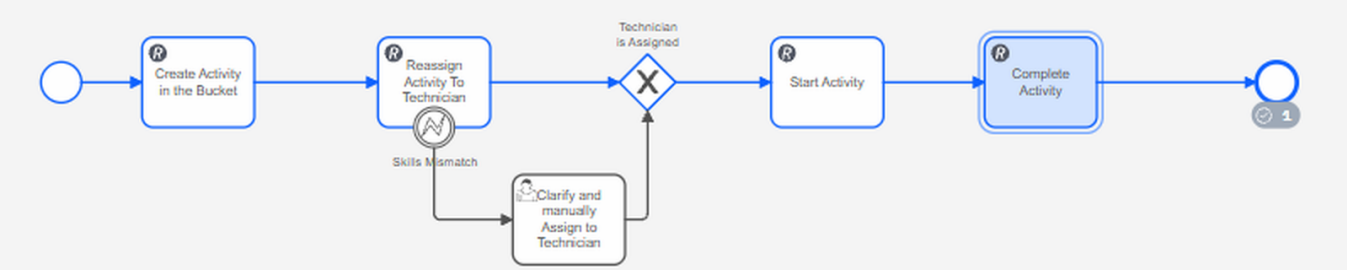
Author: Oleksiy Milner · LinkedIn · GitHub repository

The case-study process is not only drawn, it also runs. In this proof of concept, a Camunda 8 process executes as real REST calls against a mock service built from the OpenAPI spec, and the data changes in a database. It covers a happy path and a business-rule failure path. All names and data are illustrative; no real company data is used.

What is here

Item	Purpose
`Fix_Internet_problems__REST_API_Error_processing__.bpmn`	Executable Camunda 8 model: four REST service tasks, an error boundary event and a manual user task
`mock-api/`	FastAPI + SQLite service implementing `POST`, `GET` and `PATCH /activities` from the OpenAPI spec
`screenshots/`	Evidence of both runs

How it works



Task	REST call	Effect
Create Activity in the Bucket	`POST /activities` with `activityType`, `date`, `resourceId`	A new activity is inserted with status pending in the bucket. The activityId from the response is stored as a process variable.
Reassign Activity To Technician	`PATCH /activities/{activityId}` with `resourceId` from the variable `technicianId`	The activity is reassigned. On `422` (technician lacks the required work skill), an error expression raises a BPMN error and the Skills Mismatch boundary event routes to the manual task.
Start Activity	`PATCH /activities/{activityId}`	Status is updated to started.
Complete Activity	`PATCH /activities/{activityId}` with `status: completed`	Status is updated to completed.
Clarify and manually Assign to Technician	none (user task in Tasklist)	A dispatcher handles the mismatch manually.

Process inputs

The instance is started with two variables:

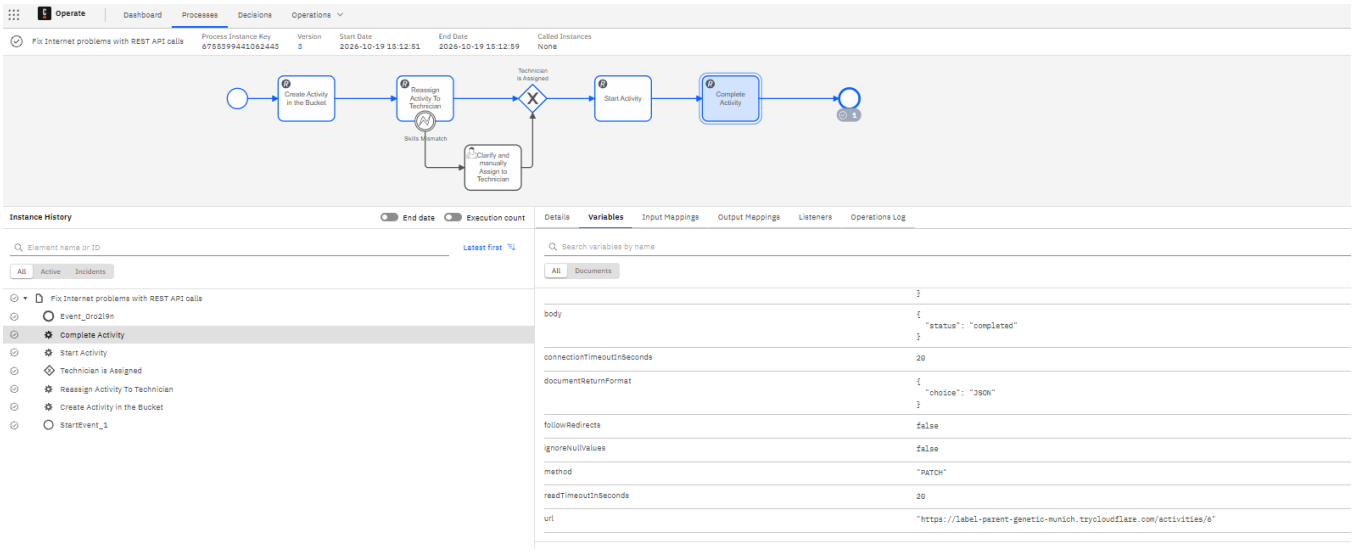
```
{"baseUrl": "https://your-tunnel.trycloudflare.com", "technicianId": 31}
```

- `baseUrl` is the address of the mock service, so the URL is not stored in the model (no trailing slash).
- `technicianId` selects the path: 31 (Technician A, has the required skill) gives the happy path; 32 (Technician B, lacks it) gives the failure path.

Evidence

Happy path (technicianId 31)

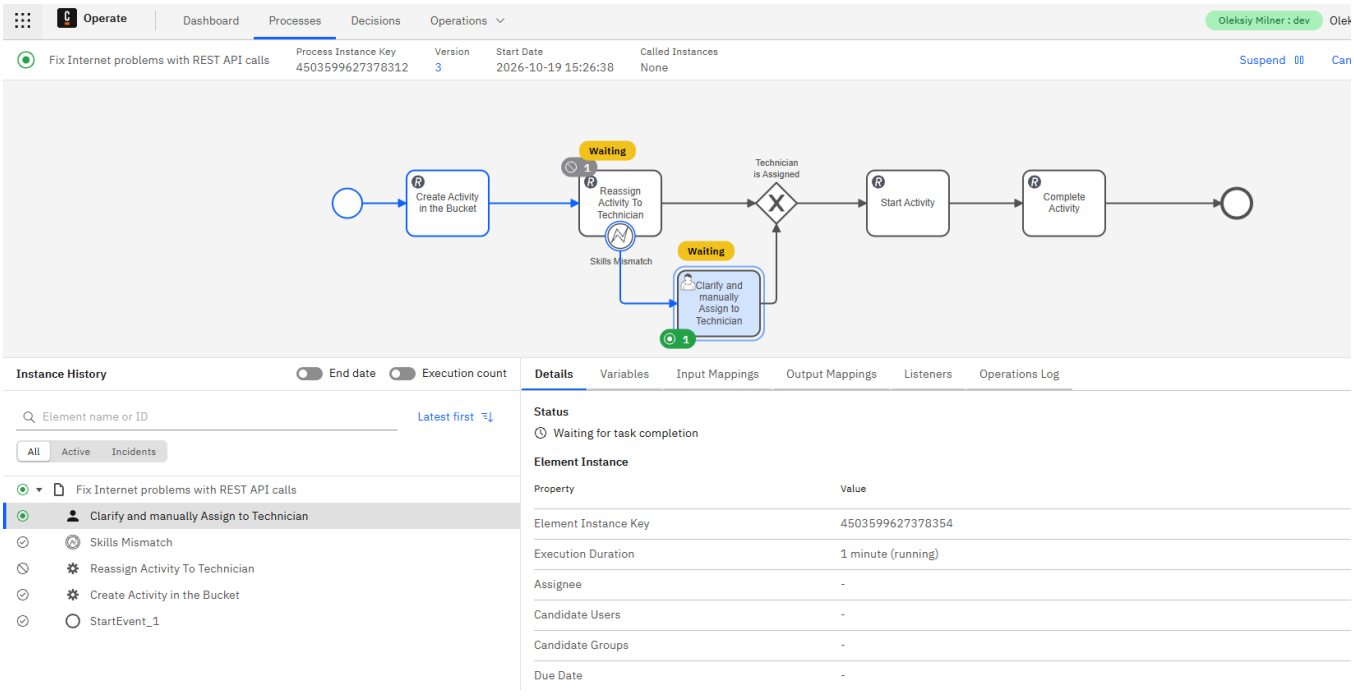
The instance completes. The service log shows one POST and three PATCH calls on the same activity.




```
INFO:     34.40.38.144:0 - "POST /activities HTTP/1.1" 201 Created
INFO:     34.40.101.119:0 - "PATCH /activities/6 HTTP/1.1" 200 OK
INFO:     35.246.141.52:0 - "PATCH /activities/6 HTTP/1.1" 200 OK
INFO:     35.246.141.52:0 - "PATCH /activities/6 HTTP/1.1" 200 OK
```

Failure path (technicianId 32)

The reassignment returns 422, the Skills Mismatch boundary event fires, and the process waits at the manual task with no incident. The two PATCH 200 calls that follow in the log come from Start and Complete Activity, after the manual task was completed.




```
INFO:     34.40.38.144:0 - "POST /activities HTTP/1.1" 201 Created
INFO:     35.246.141.52:0 - "PATCH /activities/7 HTTP/1.1" 422 Unprocessable Entity
INFO:     35.234.93.169:0 - "PATCH /activities/7 HTTP/1.1" 200 OK
INFO:     34.40.101.119:0 - "PATCH /activities/7 HTTP/1.1" 200 OK
```

Database after both runs



- Activity 6 (happy path): completed, assigned to technician 31.
- Activity 7 (failure path): completed, still assigned to resource 25 (the bucket). See the known simplification below.

Run it yourself

1. **Start the mock service.** Follow the README in `mock-api/` (Python 3.12 or 3.13). Set the token the process will send before starting:

   ```
   $env:DEMO_TOKEN="your-token"
   ```

2. **Make the service reachable.** Camunda SaaS cannot call localhost, so expose the service with a temporary tunnel, for example:

   ```
   cloudflared tunnel --url http://localhost:8000
   ```

3. **Set the token in the model.** The exported model contains the placeholder `{{BEARER_TOKEN}}`. In your own cluster, create a connector secret and reference it as `{{secrets.<NAME>}}` in the authentication field of each task. The secret value must equal the service token.
4. **Start an instance.** In Web Modeler, open the Implement tab, click Run, and enter the variables JSON shown above.
5. **Check the result** in Operate and in the database:

   ```
   python -c "import sqlite3; [print(r) for r in sqlite3.connect('outage.db').execute('select * from activity')]"
   ```

Limitations and next steps

- The bearer-token check is a static token, not real JWT validation. The tunnel is temporary and the database is a single SQLite file; this is a demo setup, not a deployment.
- **Known simplification:** the manual task does not assign a technician, so after it the process still starts and completes the activity while it remains in the bucket (activity 7 above). A fuller design would loop back to the reassignment task with a technician chosen by the dispatcher.
- The Skills Mismatch boundary event has no error code attached, so it catches any BPMN error raised by that task. The error expression raises one only for `422`.
- The executable model covers a reduced flow. SLA check, the 30-minute escalation timer, follow-up visit and customer confirmation stay in the full diagram in the main case-study repository.
- Planned: decision tables for the SLA check and technician matching.

Tools

Camunda 8 SaaS (Web Modeler, Operate, Tasklist, REST Outbound Connector), FastAPI, SQLite, Cloudflare quick tunnel.